

Developing Graphics Library with DirectFB

Deals with commonly used software layering in between the DirectFB software library and a graphics card for graphics acceleration in embedded systems

■ BHAVIN PATEL

Living in a world full of graphics and multimedia gadgets, Linux embedded engineers must know ins and outs of multimedia-capable embedded Linux systems.

DirectFB is one of the widely used graphics libraries that provides hardware graphics acceleration. Obviously, it does not run on the hardware directly as it is a library independent of hardware. Graphics developers use certain software layering in between DirectFB and hardware to let DirectFB access the hardware.

Here we describe commonly used graphics library scenarios between a graphics card and DirectFB. However, before jumping on graphics library layering, a brief is given about DirectFB for those who are not much aware of it.

About DirectFB

DirectFB is a user-space light-weight library (smaller than 700 kB in size) developed for special multimedia requirements in embedded devices. It provides hardware graphics acceleration, input device handling and abstraction, windowing system with support for translucent windows and multiple display layers.

Many a time, the 'FB' word attached with DirectFB confuses new users into believing that DirectFB runs on a Linux framebuffer device. But DirectFB can run on top of any hardware abstraction layer. It is a complete hardware abstraction layer with software fallbacks for every graphics operation that is not supported by the underlying hardware.



DirectFB adds graphical power to embedded systems and sets a new standard for graphics under Linux.

Widely used scenarios for graphics library

Chipset driver. A chipset driver is a user-space code which should be registered with the DirectFB graphics core. It is dependent on the graphics chip. For example, DirectFB has an Open Source chipset driver available for graphics accelerators such as Renesas' SH772, Texas Instruments' OMAP 3530 and DaVinci, Cirrus Logic's ep9x and NVIDIA. Chipset drivers are located at `DirectFB-<version>/gfxdriver/*`

A chipset driver should be implemented as a `gfxdriver`, which is a way

to register customised graphics hardware with the DirectFB graphics core. The chipset driver maps or unmaps the graphics card register space using the application programming interfaces (APIs) of DirectFB as follows:

```
dfb_gfxcard_map_mmio
dfb_gfxcard_unmap_mmio
```

The chipset driver implements hardware-specific read/write functions to directly access graphics hardware registers.

As shown in Fig. 2, no kernel-level implementation is required in this case. While configuring DirectFB, the user needs to use '`-gfxdriver=<mychipset>`' option. It will hook the chipset driver with the DirectFB system. When DirectFB is executed, it probes for avail-